

M17 TEACHING BOOK

Architecture & System Design

THE LEARNING LOOP

- See the problem before the product.
- Follow the teacher reasoning once.
- Do the changed problem yourself.
- Validate evidence, not confidence.
- Repair only after a saved attempt.
- Defend trade-offs and route to G06.

BEGINNER-FIRST PROFESSIONAL INSTITUTE REBUILD

Preserves the authoritative 10-lesson M17 curriculum; expands teaching depth, routes, and evidence controls.

This book is your teacher

How to use it

Assume architecture/system-design reasoning is new even though M15-M16 taught the component technologies. Every new decision word is introduced before you must use it. Read one lesson in order; do not jump to review/retest because the answer will feel familiar after you see it.

The normal loop

1) Read why/goal. 2) Picture the decision. 3) Learn new words. 4) Use the mapped visual/reference if useful. 5) Follow the BookBox teacher example once. 6) Close help. 7) Solve MetroMart. 8) Validate. 9) Explain back. 10) Save evidence. 11) Only if weak: review -> changed CareLink micro-proof -> reassessment.

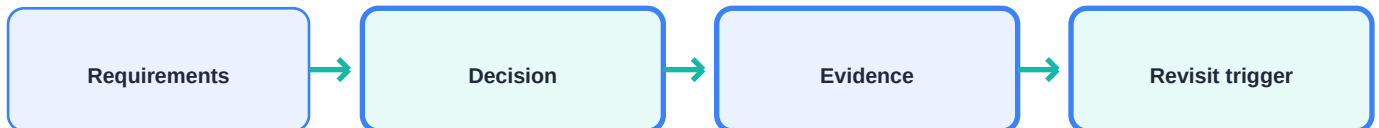
Truth boundary

Architecture diagrams and JSON fixtures are design evidence. The local checker proves structural completeness only. It does NOT prove Fabric, cloud, network, IAM, failover, or production runtime behavior. Never convert a simulation into a runtime claim.

Contents / Index

Orientation: module map / tools / routes	4-5
DE17-01 From requirements to architecture	6
DE17-02 Batch vs streaming decisions	11
DE17-03 Warehouse vs lakehouse	16
DE17-04 Managed service vs build yourself	21
DE17-05 Scaling and bottleneck reasoning	26
DE17-06 Reliability and failure domains	31
DE17-07 Cost-aware architecture	36
DE17-08 Security and governance by design	41
DE17-09 Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes	46
DE17-10 Architecture review and defense	51
End-of-module mastery checklist	56
Revision / architecture quick reference	57
G06 handoff: exact next route	58

Module map



Canonical sequence

DE17-01 requirements -> 02 batch/stream -> 03 warehouse/lakehouse -> 04 managed/build -> 05 scale/bottlenecks -> 06 reliability/failure domains -> 07 cost -> 08 security/governance -> 09 promotion foundations -> 10 architecture defense.

First-use tool map

Read-only masters: `metromart\_requirements.json`, `carelink\_requirements.json`, `fleetstream\_requirements.json`, `architecture\_decision\_template.json`.

FILL: a copied ADR template, never the master.

CREATE: requirement trace, diagram, validation notes, repair note, changed retry, defense notes.

RUN: `python evaluate\_evidence.py your\_copy.json` for structural completeness only.

Learning domains

BookBox = teacher demonstration only. MetroMart = independent lesson proof. CareLink = changed retry/retest. FleetStream = module practical. Keeping domains different protects transfer and reduces memorization.

Exact learner workspace route

CREATE vs FILL

CREATE means start a new learner evidence file from a blank page/file. **FILL** means copy the provided blank template into your learner workspace, then fill your copy. Never type answers into the authoritative requirement files or master template.

Evidence paths

```
`learner_workspace/01_ATTEMPTS/METROMART/DE17-XX/attempt_01.md`  
`02_ADR/metromart_architecture_decision.json`  
`03_DIAGRAMS/metromart_architecture.*`  
`04_VALIDATION/metromart_validation.md`  
`05_REPAIR/DE17-XX/repair_note.md` (only after saved weak attempt)  
`06_RETRY/CARELINK/DE17-XX/retry_01.md` (only if assigned)  
`07_PRACTICAL/FLEETSTREAM/`
```

Protected failure route

SAVED ATTEMPT -> targeted teaching repair -> CHANGED micro-proof -> validation -> reassessment. Original attempts remain append-only. A review never authorizes editing history into a perfect first try.

No score shortcut

Mastery is competency/evidence based. Do not average weak architecture, security, reliability, or defense into a pass. Each required competency must be defensible.

DE17-01 From requirements to architecture

1. Why this matters

A data engineer is rarely handed a perfect diagram. You are handed business goals, messy constraints, data sources, freshness targets, security rules, recovery needs, and cost limits. Architecture begins by turning those facts into explicit decisions.

2. What you are learning

Translate functional and non-functional requirements into an architecture decision trail. Separate what the system must do from the qualities it must protect, then record decisions in an ADR/evidence packet.

3. Prerequisite / start state

M15 cloud/Fabric and M16 production controls are complete. You already know pipelines, storage, security, monitoring, and delivery. Here you learn how to choose and defend how those pieces fit together.

4. Picture it in your head

Picture a doctor reading a case sheet before prescribing. The symptom is not the treatment. Requirements are the case sheet; architecture is the reasoned prescription. Every major box in the diagram should point back to a requirement.

5. New words before we use them

Functional requirement - A capability the system must provide, such as ingest orders or produce an alert.

Non-functional requirement - A quality/constraint such as freshness, availability, RTO/RPO, security, scale, or cost.

ADR - Architecture Decision Record: decision, context, alternatives, trade-off, evidence, and revisit trigger.

Constraint - A boundary you must respect: team size, region, budget, compliance, or existing system.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward. Use the cloud example to see trade-off language; our lesson generalizes beyond one platform.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Microsoft Azure Architecture Center - Architecture guide

<https://learn.microsoft.com/en-us/azure/architecture/guide/>

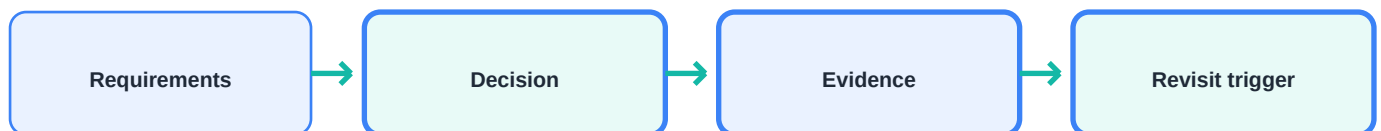
Why use it: Use for current requirement-to-design and architecture-quality guidance.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox needs daily finance reporting and near-real-time low-stock alerts. We do not start by saying "use streaming everywhere." We write two flows, their freshness, consumers, correctness needs, failure tolerance, security, and cost. Only then do we choose different paths where the requirements differ.



8. Expected result

What a correct-looking result contains

A requirement inventory that names business goal, each flow, freshness/correctness, restricted data, RTO/RPO, scale, cost guardrails, and team/operational constraints - plus at least three traceable decisions.

9. Why it worked

Reasoning

The choices can be challenged because each has a visible reason. When a requirement changes, you know which decision to revisit instead of redrawing the whole system blindly.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Choosing products before writing requirements.
- Treating "real time" as a requirement without a number.
- Hiding budget/team limits because they feel non-technical.
- Recording a decision but no rejected alternative or revisit trigger.

11. Stuck / recovery help

Recovery ladder

If you feel stuck, stop drawing. Make a two-column sheet: "What must be true?" and "How will we prove it?" Convert vague words into measurable facts, then make one decision at a time.

12. Small independent task

MetroMart - close teacher example first

MetroMart: read ``metromart_requirements.json``. CREATE a requirement-to-decision trace for the three flows. FILL a copy of ``architecture_decision_template.json``; do not edit the master template.

Save before help

Save ``attempt_01.md`` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Every decision cites at least one requirement; no requirement silently disappears; the architecture can explain why orders, inventory alerts, and clickstream may need different treatment.

14. Teach it back

Explain without notes

Explain in 60-90 seconds why "use a lakehouse" is not an architecture until requirements, alternatives, and operating evidence are attached.

15. Protected review + changed retry

Failure / repair route

Only after a saved MetroMart attempt may review be opened. Repair only the weak requirement/decision link, then solve a changed CareLink micro-proof without copying the original wording.

16. Evidence / mastery + exact next route

Evidence

Save the original MetroMart attempt append-only, the requirement trace, copied ADR JSON, one diagram, validation notes, and a short decision defense. PASS -> DE17-02. Weak/uncertain -> targeted repair + changed proof.

DE17-02 Batch vs streaming decisions

1. Why this matters

Batch and streaming are not status symbols. They are ways of meeting different freshness, ordering, replay, and operating needs. The wrong choice can make a simple workload expensive and fragile.

2. What you are learning

Choose batch, streaming, or a deliberate hybrid per data flow. Explain latency, bounded/unbounded data, event time, replay, ordering, idempotency, and the condition that would make you revisit the decision.

3. Prerequisite / start state

DE17-01 requirement trace exists. You know basic Kafka/streaming from M14 and cloud pipelines from M15.

4. Picture it in your head

Think of postal mail versus a live phone call. Some messages are fine in a daily bundle; some decisions lose value if they arrive late. A real company often needs both.

5. New words before we use them

Batch - Process a bounded collection on a schedule or trigger.

Streaming - Process an ongoing event flow with low latency.

Event time - When the business event happened, which may differ from processing time.

Replay - Reprocess retained input after a bug/failure.

Idempotent sink - Repeated delivery does not create repeated business effect.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward. Watch only for decision examples; then close it.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Azure Architecture Center - Big data architecture style

<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/big-data>

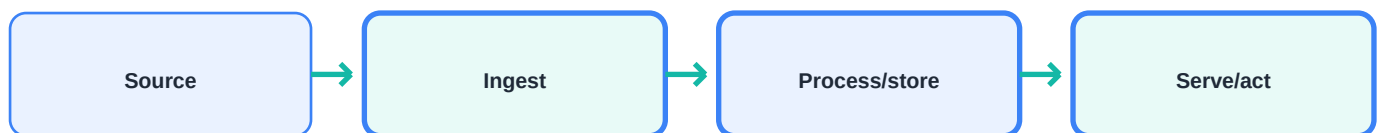
Why use it: Current batch/real-time architecture reference.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox finance closes daily, so a scheduled batch is simple and auditable. Low-stock alerts must react within minutes, so a streaming/event path may be justified. Both can land durable raw data so replay and later reconciliation are possible.



8. Expected result

What a correct-looking result contains

One explicit mode decision per flow, with freshness target, delivery semantics, replay strategy, ordering need, sink behavior, operational burden, and a changed-constraint trigger.

9. Why it worked

Reasoning

The architecture spends streaming complexity only where the business value requires it, while preserving a durable path for correction and audit.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Streaming everything because it sounds modern.
- Calling a 30-minute requirement "real time."
- Ignoring duplicate delivery and external side effects.
- No replay plan, so a code defect becomes permanent data loss.

11. Stuck / recovery help

Recovery ladder

Write the freshness target first. Ask what breaks if data is 5 minutes, 30 minutes, or 4 hours late. Then ask whether duplicate events are safe and whether the source can be replayed.

12. Small independent task

MetroMart - close teacher example first

MetroMart: decide mode for `orders\_finance`, `inventory\_alerts`, and `clickstream`. CREATE a tiny table: flow | mode | freshness | replay | duplicate strategy | revisit trigger.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

A 30-minute finance SLA does not require sub-second streaming; the 2-minute alert path addresses at-least-once delivery with an idempotent sink; clickstream choice respects the 4-hour target.

14. Teach it back

Explain without notes

Explain why "streaming is faster" is insufficient. Include operational complexity, correctness, replay, and business value.

15. Protected review + changed retry

Failure / repair route

After saved attempt -> review -> changed CareLink decision: appointments API vs claims files. Prove why the changed freshness/correctness constraints alter or preserve your choice.

16. Evidence / mastery + exact next route

Evidence

Append mode table and ADR decision. Evidence must state the rejected alternative and one observable signal that would trigger a redesign. PASS -> DE17-03.

DE17-03 Warehouse vs lakehouse

1. Why this matters

Warehouse and lakehouse can overlap, but their strengths differ. A professional design chooses a serving/storage pattern from data shape, consumers, governance, concurrency, openness, and transformation needs - not from a product label.

2. What you are learning

Compare warehouse, lakehouse, and combined patterns. Define raw, curated, and serving ownership so there is one understandable source of truth rather than duplicated pipelines.

3. Prerequisite / start state

DE17-02 mode decisions are recorded. You know Delta/lakehouse from M13 and Fabric warehouse/lakehouse concepts from M15.

4. Picture it in your head

Picture a workshop plus a storefront. The workshop handles varied raw materials and heavy transformation; the storefront is optimized for predictable customer access. A lakehouse/warehouse design can separate those responsibilities while sharing governed truth.

5. New words before we use them

Lakehouse - Data-lake openness plus table/transaction capabilities for analytical engineering.

Warehouse - Structured analytical store optimized for governed SQL analytics and serving.

Serving layer - The consumer-facing representation optimized for a known access pattern.

One source of truth - Clear ownership so two stores do not silently become competing realities.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Fabric analytical data store technology choices

<https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/fabric-analytical-data-stores>

Why use it: Compare workload, ingestion, access, and serving needs instead of choosing by fashion.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox can retain raw/replayable events in lakehouse storage, transform to curated tables, then expose a SQL-friendly serving model where finance users need stable relational semantics. This is a boundary decision, not a rule that every system needs two stores.

8. Expected result

What a correct-looking result contains

A store choice per stage with data type/volume, compute pattern, ingestion mode, query access, consumer, governance, concurrency, portability, and duplication risk.

9. Why it worked

Reasoning

The store is justified by workload. Raw replay and engineering flexibility remain clear, while governed SQL serving is not forced to behave like raw storage.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- "Lakehouse replaces everything" as a slogan.
- Copying the same curated truth into multiple stores without ownership rules.
- Ignoring SQL concurrency/BI behavior.
- Choosing open formats but forgetting security/governance responsibilities.

11. Stuck / recovery help

Recovery ladder

List consumers and their access patterns. Mark which data must be replayable, which needs heavy Spark-style engineering, and which needs predictable SQL serving. Then choose the smallest coherent set of stores.

12. Small independent task

MetroMart - close teacher example first

MetroMart: CREATE a raw -> curated -> serving map. For each stage state warehouse/lakehouse/combined choice and why. Identify one duplication risk and how ownership prevents it.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Every stage has a named owner/purpose. Finance totals can reconcile to curated truth. Raw data remains available for replay where required. No duplicate store is justified only by habit.

14. Teach it back

Explain without notes

Explain to a manager why warehouse vs lakehouse is a workload decision, not a winner/loser debate.

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: decide where regulated raw claims, latest appointments state, and de-identified facility aggregates belong. Do not reuse MetroMart labels without checking constraints.

16. Evidence / mastery + exact next route

Evidence

Save the stage map, one reconciliation path, rejected alternative, and revisit trigger. PASS -> DE17-04.

DE17-04 Managed service vs build yourself

1. Why this matters

Building everything yourself gives control but also transfers patching, upgrades, scaling, backup, on-call, and security responsibility to your team. Managed services reduce some burden but introduce limits, service boundaries, and provider dependency.

2. What you are learning

Make managed-vs-build decisions with total operational cost, control, portability, compliance, team capacity, failure responsibility, and exit strategy.

3. Prerequisite / start state

You have a storage/serving boundary from DE17-03 and know managed Fabric/cloud services from M15.

4. Picture it in your head

Owning a taxi versus booking one. Ownership gives control but you maintain the vehicle. A managed service is not "free operations"; it changes which responsibilities are yours.

5. New words before we use them

Shared responsibility - Provider and customer own different parts of reliability/security/operations.

Operational burden - Human and technical work needed to patch, scale, monitor, recover, and support a system.

Exit plan - How you would migrate if cost, limits, regulation, or strategy changes.

TCO - Total cost of ownership: service cost plus people, incidents, maintenance, and opportunity cost.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Azure Architecture design principles

<https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/>

Why use it: Use to connect operational burden, reliability, and service boundaries.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox has a small platform team. A managed ingestion/orchestration service may be worth more than a cheaper self-hosted cluster because on-call and upgrades would consume the team. A specialized self-managed component is justified only if a requirement cannot be met otherwise.

8. Expected result

What a correct-looking result contains

For each major service boundary: managed/build choice, responsibility split, control gained/lost, team impact, cost driver, lock-in risk, and exit/revisit trigger.

9. Why it worked

Reasoning

The decision accounts for the people operating the architecture, not only the diagram. That is what turns a lab design into a production design.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Comparing only monthly cloud price.
- Assuming managed means no incidents.
- Self-hosting because it looks more "engineering."
- No exit path for a proprietary boundary.

11. Stuck / recovery help

Recovery ladder

Write the recurring duties: patch, backup, scale, monitor, secure, recover, upgrade, support. Put an owner beside each. If the owner column is vague, your build-yourself choice is not finished.

12. Small independent task

MetroMart - close teacher example first

MetroMart: team size is four and managed services are preferred. CREATE a responsibility table for ingestion, compute, storage, orchestration, and secrets. Choose at least one rejected build-yourself alternative.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Every operational duty has an owner; no service is "managed" without customer responsibilities; the plan fits a four-person team and names a lock-in/exit risk.

14. Teach it back

Explain without notes

Explain why a more expensive managed service can still have lower TCO for a small team.

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: regulation/region constraints are stronger. Re-evaluate one managed boundary; show exactly which new constraint changes the decision, or justify why it does not.

16. Evidence / mastery + exact next route

Evidence

Save responsibility/TCO reasoning and ADR update. PASS -> DE17-05.

DE17-05 Scaling and bottleneck reasoning

1. Why this matters

Scale problems are usually bottleneck problems, not "add more servers" problems. You need a measurable signal, the constrained resource, a scaling dimension, and proof that the change helped without moving the failure elsewhere.

2. What you are learning

Reason about CPU, memory, I/O, network, partition skew, concurrency, quotas, small files, and downstream limits. Choose vertical/horizontal/partition scaling with load-test evidence and rollback/revisit triggers.

3. Prerequisite / start state

Architecture service boundaries exist. You know Spark/partitioning from M12-M13 and cloud monitoring from M15-M16.

4. Picture it in your head

A highway jam does not disappear if you widen a road after the toll booth when the toll booth is the bottleneck. Find the narrowest point first.

5. New words before we use them

Bottleneck - The constrained resource limiting end-to-end throughput/latency.

Vertical scale - Give one unit more CPU/memory/capacity.

Horizontal scale - Add parallel workers/partitions/instances.

Skew - Uneven data/work distribution that leaves some workers overloaded.

Backpressure - Downstream slowness forces upstream processing to slow/queue.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Azure Architecture design principles

<https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/>

Why use it: Use as current scaling/resiliency design overlay.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox holiday traffic rises 4x. We measure queue lag, compute utilization, partition balance, storage I/O, and sink throughput. If one hot partition owns most traffic, adding workers alone may not help; repartitioning/key design may be the real fix.

8. Expected result

What a correct-looking result contains

A bottleneck hypothesis with signal, threshold, scaling dimension, 4x/10x test, downstream risk, and post-change validation.

9. Why it worked

Reasoning

It connects a change to an observed constraint and validates the whole path, preventing expensive scale that does not solve the real limit.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Scaling before measuring.
- Watching average CPU while one partition is overloaded.
- Ignoring quotas/connections on the downstream sink.
- No load shape that resembles real peaks.

11. Stuck / recovery help

Recovery ladder

Draw the path left to right. At each hop write one capacity signal and queue/latency signal. The first place where demand exceeds sustainable service is your leading bottleneck hypothesis.

12. Small independent task

MetroMart - close teacher example first

MetroMart: model the 4x festival case. CREATE a bottleneck table with signal, likely limit, scale action, trigger, and validation for ingestion, compute, storage, and serving.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

At least one skew/concurrency/downstream scenario is considered; triggers are measurable; the scale plan names how you know the fix worked and whether cost stayed acceptable.

14. Teach it back

Explain without notes

Explain why "autoscale" is not a complete scaling strategy.

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: acquisition doubles clinics and introduces incompatible schemas. Identify one throughput bottleneck and one schema/contract bottleneck; propose different controls.

16. Evidence / mastery + exact next route

Evidence

Save scale plan and observed/expected signals. PASS -> DE17-06.

DE17-06 Reliability and failure domains

1. Why this matters

Production systems fail in pieces: a region disconnects, a source duplicates, a checkpoint corrupts, a secret expires, or a sink slows. Reliability comes from containing blast radius and proving recovery, not from saying "high availability."

2. What you are learning

Identify failure domains, blast radius, RTO/RPO, durable replay points, idempotency, recovery sequence, and reconciliation evidence.

3. Prerequisite / start state

DE17-05 bottleneck map exists. You know incidents/runbooks/observability from M16.

4. Picture it in your head

A ship has watertight compartments. Damage to one compartment should not sink the whole ship. Failure domains are the compartments of your data platform.

5. New words before we use them

Failure domain - A component/region/dependency that can fail independently.

Blast radius - How much of the system/business is affected by one failure.

RTO - Target time to restore service.

RPO - Maximum acceptable data loss window.

Recovery validation - Evidence that restored data/service is complete and correct, not merely running.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Azure Well-Architected Reliability

<https://learn.microsoft.com/en-us/azure/well-architected/reliability/>

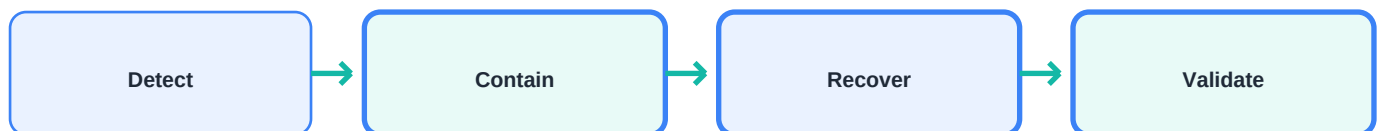
Why use it: Current reliability and recovery guidance.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox alert processing stops. Durable raw events and checkpoints let the stream resume; idempotent sink logic makes replay safe. A runbook restores processing, then reconciliation compares input IDs, processed IDs, and business side effects before closing the incident.



8. Expected result

What a correct-looking result contains

For important failures: detect -> contain -> recover -> replay -> reconcile -> validate, tied to RTO/RPO and named evidence.

9. Why it worked

Reasoning

It assumes failure will happen and makes recovery observable. The design protects correctness during replay instead of trading data loss for uptime.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- RTO/RPO written but no mechanism supports them.
- Backups never restored in a test.
- Replay duplicates alerts/payments.
- One shared dependency creates a hidden global blast radius.

11. Stuck / recovery help

Recovery ladder

Pick one failure at a time. Ask: what stops, what remains durable, how do we detect it, what is the first recovery action, what can duplicate, and what evidence proves the business state is correct?

12. Small independent task

MetroMart - close teacher example first

MetroMart: cover inventory path outage, raw-store unavailability, and downstream alert sink slowdown. CREATE a failure table using the template fields: failure/blast_radius/detect/contain/recover/validate.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Inventory path plan can plausibly support 60-minute RTO and 15-minute RPO; replay is safe; recovery includes reconciliation, not just service restart.

14. Teach it back

Explain without notes

Explain the difference between "the job is green" and "the business data is recovered."

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: scheduling feed requires 99.95%, RTO 30, RPO 5. Design a changed recovery micro-proof and identify the evidence you would demand after failover.

16. Evidence / mastery + exact next route

Evidence

Save failure-domain table, one recovery drill plan, and reconciliation evidence definition. PASS -> DE17-07.

DE17-07 Cost-aware architecture

1. Why this matters

Cost is an architecture constraint like latency or reliability. Good engineering controls cost without quietly breaking business value or SLOs.

2. What you are learning

Identify dominant cost drivers, unit economics, retention/tiering, autoscaling/compute windows, query/scan behavior, and guardrails with explicit trade-offs.

3. Prerequisite / start state

You have scale and reliability plans. You know cloud cost awareness from M15 and operational metrics from M16.

4. Picture it in your head

A water bill is easier to control if you know which tap is running. "Cloud is expensive" is vague; bytes stored, bytes scanned, compute time, data movement, and concurrency are measurable taps.

5. New words before we use them

Cost driver - A measurable behavior that materially changes spend.

Unit economics - Cost per useful unit: per million events, per TB processed, per customer, etc.

Guardrail - A limit/alert/policy that catches waste before it becomes a surprise.

FinOps - Engineering/finance practice of making cloud value and cost visible and accountable.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Azure Well-Architected Cost Optimization principles

<https://learn.microsoft.com/en-us/azure/well-architected/cost-optimization/principles>

Why use it: Current cost/prioritization guidance.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox sets cost-per-million-events and daily compute-hour expectations. Raw data uses deliberate retention/tiering; expensive queries are monitored; burst capacity is allowed when SLO value justifies it. The response to a spike is diagnosis, not blind throttling.

8. Expected result

What a correct-looking result contains

At least three cost drivers, two guardrails, a normal vs peak envelope, one unit metric, and a trade-off statement linking cost action to SLO/business impact.

9. Why it worked

Reasoning

It separates useful peak spend from waste and gives the team an early warning before a bill becomes the only signal.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Cutting compute until freshness fails.
- No ownership for unused storage/idle capacity.
- Ignoring scan/data-egress costs.
- Using only a total monthly bill with no unit metric.

11. Stuck / recovery help

Recovery ladder

Rank resources by estimated spend/usage. For the top three, write "what makes this number go up?" and one engineering control. Then test whether the control harms freshness, reliability, or compliance.

12. Small independent task

MetroMart - close teacher example first

MetroMart: normal-month target ≤ 100 relative units; festival bursts above 140 require justification. CREATE 3+ drivers, 2+ guardrails, unit metric, and a burst decision rule.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Guardrails are measurable and have owners; peak spend can be justified by required capacity/value; no cost action violates the inventory SLO silently.

14. Teach it back

Explain without notes

Explain why the cheapest architecture can be the most expensive business outcome.

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: target ≤ 70 unless compliance/reliability evidence justifies more. Choose one costly control you would keep and one cost you would reduce, with evidence.

16. Evidence / mastery + exact next route

Evidence

Save cost plan and decision trade-off. PASS -> DE17-08.

DE17-08 Security and governance by design

1. Why this matters

Security added after architecture becomes a patchwork. Data classification, identity, least privilege, protection, lineage, retention, and audit should shape boundaries before production.

2. What you are learning

Design security/governance controls around data sensitivity and trust boundaries. Use workload identity, least privilege, encryption, masking/tokenization, lineage, retention, audit, and separation of duties.

3. Prerequisite / start state

You know IAM, secrets, PII, metadata, lineage and governance from M15-M16. Here you integrate those controls into the architecture.

4. Picture it in your head

An office has public lobby, staff rooms, finance vault, and visitor log. Data architecture also needs zones, identities, permissions, and records of who touched sensitive assets.

5. New words before we use them

Data classification - Label data by sensitivity/business handling needs.

Workload identity - A machine/service authenticates as itself instead of using a human password.

Least privilege - Grant only the actions and scope required.

Lineage - Trace where data came from, how it changed, and where it went.

Trust boundary - Point where identity/network/data assumptions change and controls must be explicit.

6. Useful video / source

Mapped source

VIDEO SUPPORT: DP-700 Full Course - Architectural Decision-Making

<https://www.youtube.com/watch?v=KiB4eAeFRsw>

Why use it: 2:19:27 onward.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Azure Well-Architected Security checklist

<https://learn.microsoft.com/en-us/azure/well-architected/security/checklist>

Why use it: Current security-by-design checklist overlay.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox customer contact fields are restricted while regional/category aggregates are broadly consumable. Pipelines use workload identities; secrets are not in code; raw sensitive data has narrow access; curated/serving layers expose only needed fields; lineage and audit connect access to transformations.

8. Expected result

What a correct-looking result contains

A classification/access matrix, identity boundary, protection method, audit evidence, lineage/retention owner, and response to unauthorized/incorrect access.

9. Why it worked

Reasoning

Controls follow the data through the system. Broad analytics can remain useful without giving every consumer raw sensitive fields.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- One shared admin identity.
- Secrets committed to code/config.
- Masking in a dashboard while raw export stays open.
- Lineage exists but no owner/review process.

11. Stuck / recovery help

Recovery ladder

Start with the most sensitive field. Trace every place it can travel. At each hop ask: who needs it, why, for how long, how is access proven, and what safer representation could serve most users?

12. Small independent task

MetroMart - close teacher example first

MetroMart: customer_email and phone are restricted; region/category aggregates are broad. CREATE a control matrix for raw, curated, serving, logs, and backups. Include audit evidence and retention owner.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Restricted data has narrower scope than aggregates; no secret is embedded; access is identity-based; lineage/audit can answer who/what/when; backups are included.

14. Teach it back

Explain without notes

Explain why encryption alone does not provide least privilege or governance.

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: patient_name, email and diagnosis_code are restricted; only de-identified facility aggregates are broad. Build one changed access/lineage proof.

16. Evidence / mastery + exact next route

Evidence

Save security_controls + governance sections of ADR, plus a trust-boundary diagram note. PASS -> DE17-09.

DE17-09 Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes

1. Why this matters

A promotion-ready engineer should recognize advanced building blocks without treating them as compulsory badges. CDC, data contracts, Vault, data mesh, and Kubernetes solve different problems and can add serious operational cost when used without a requirement.

2. What you are learning

For each concept: define the problem, identify when it is useful, when to reject it, operational risks, and a smaller alternative. This is architecture literacy, not five separate certifications.

3. Prerequisite / start state

DE17-01 through DE17-08 are complete. You already know Kafka, cloud, security, CI/CD, and operations foundations.

4. Picture it in your head

Think of a toolbox. A hammer is valuable when you have a nail; carrying five power tools to hang one picture creates more problems than it solves.

5. New words before we use them

CDC - Capture inserts/updates/deletes from a source database change log.

Data contract - Producer-consumer agreement on schema/semantics/compatibility and ownership.

Vault - Secrets-management platform for identity-driven secret lifecycle/dynamic credentials.

Data mesh - Socio-technical operating model: domain ownership, data as product, self-serve platform, federated governance.

Kubernetes - Container orchestration for deployment, scaling, healing, and configuration; not a default data platform.

6. Useful video / source

Mapped source

OFFICIAL: Debezium CDC architecture

<https://debezium.io/documentation/reference/stable/architecture.html>

Why use it: CDC components and change-event mental model.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

VIDEO SUPPORT: CDC Use Case & System Design

https://www.youtube.com/watch?v=dN_11nBcv_A

Why use it: Preserved from the authoritative local M17 visual bundle; use only as an optional visual, with Debezium docs as current fallback.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

VIDEO SUPPORT: Confluent - Data Contracts for Long-Term Schema Management

<https://www.youtube.com/watch?v=ZIJ8cs-cJU>

Why use it: Useful marks: 2:02 contract definition; 8:38 Avro/Protobuf; 22:33 compatibility.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

VIDEO SUPPORT: Thoughtworks / Zhamak Dehghani - Introduction to Data Mesh

https://www.youtube.com/watch?v=_bmYXWCxF_Q

Why use it: Use for the four-principle mental model; data mesh is not a product install.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: HashiCorp Vault + Kubernetes tutorial

<https://developer.hashicorp.com/vault/tutorials/kubernetes-introduction>

Why use it: Secrets/workload-authentication awareness.

After it: close the source and return to this book; do not copy an architecture from the video.

Mapped source

OFFICIAL: Kubernetes Basics

<https://kubernetes.io/docs/tutorials/kubernetes-basics/>

Why use it: Deployment/scaling/update/debugging awareness.

After it: close the source and return to this book; do not copy an architecture from the video.

7. Follow with me once

BookBox teacher demonstration

BookBox needs reliable OLTP changes without polling: CDC may fit. Producers frequently break downstream schemas: a data contract may fit. Managed identity and cloud secret store already meet the risk: Vault may be unnecessary. One small centralized data team: data mesh may be premature. A managed data platform: Kubernetes may add more operations than value.

8. Expected result

What a correct-looking result contains

Five use-or-reject rows, each containing problem, requirement signal, operational risk, smaller alternative, and revisit trigger.

9. Why it worked

Reasoning

The reasoning prevents resume-driven architecture. Advanced tools are adopted only when their benefits exceed their new failure modes and team burden.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Calling CDC the same as streaming.
- Treating schema registry alone as a complete data contract.
- Putting Vault in the diagram without defining identity/rotation.
- Calling "teams own tables" data mesh without product/platform/governance changes.
- Using Kubernetes because containers exist.

11. Stuck / recovery help

Recovery ladder

For any unfamiliar concept, answer three questions only: What concrete pain does it solve? What new thing must we operate? What happens if we do not adopt it yet?

12. Small independent task

MetroMart - close teacher example first

MetroMart: FILL all five `advanced\_concepts` rows in the ADR. You may reject a concept. Rejection is correct when requirements do not justify its cost - but you must give a reason and revisit trigger.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Each row has use/reject + reason; at least one alternative is named; no concept is accepted only because it is "industry standard."

14. Teach it back

Explain without notes

Pick any two concepts and explain why they are not substitutes for each other.

15. Protected review + changed retry

Failure / repair route

Changed CareLink proof: incompatible schemas after acquisition make contracts more relevant; regulated secrets may change secret-management choice. Reassess without assuming every concept must become "USE."

16. Evidence / mastery + exact next route

Evidence

Save advanced_concepts decisions and source notes. PASS -> DE17-10.

DE17-10 Architecture review and defense

1. Why this matters

Architecture is not finished when the diagram looks clean. It is finished enough to act when another engineer can trace requirements to decisions, challenge alternatives, understand risks, and see the evidence/revisit conditions.

2. What you are learning

Run an architecture review and defend the design under changed constraints. Communicate top risk, rejected alternatives, failure/recovery, evidence, and what decision you would revisit first.

3. Prerequisite / start state

All previous M17 decisions and evidence exist. No tutorial is used for the independent defense.

4. Picture it in your head

Imagine a design review where a senior engineer changes one fact: traffic becomes 10x, a region disconnects, budget is cut, or a compliance rule appears. A strong architecture bends at the affected decision instead of collapsing everywhere.

5. New words before we use them

Architecture defense - Concise explanation of why the design meets requirements and what trade-offs remain.

Revisit trigger - Observable change that makes a prior decision worth reopening.

Evidence-backed review - Claims supported by tests, metrics, controls, runbooks, or explicit assumptions.

Changed constraint - A new condition used to test whether the architecture reasoning transfers.

6. Useful video / source

Mapped source

PRACTICE ONLY: Independent architecture defense

No tutorial by design. Close videos, review, and AI solving. Defend your own evidence.

7. Follow with me once

BookBox teacher demonstration

BookBox review: state goal and top two requirements, walk only the major flow/boundaries, name one rejected alternative, explain biggest failure domain and recovery, show one cost/security control, then accept a changed constraint and identify only the decisions it affects.

8. Expected result

What a correct-looking result contains

A 5-minute defense packet: one architecture diagram, complete ADR, requirement trace, failure/scale/cost/security evidence, top risk, rejected alternatives, changed-constraint response, and 60-second summary.

9. Why it worked

Reasoning

The review exposes hidden assumptions before production and proves you can reason beyond memorized products.

Exact guided route

Open the guided-labs book for this same lesson. Work on the BookBox demonstration only. Say the requirement first, then the decision, rejected alternative, trade-off, evidence, and revisit trigger. Do not open MetroMart review/retest while following the teacher demonstration.

CREATE / FILL checkpoint

Guided BookBox notes: CREATE if you want scratch notes. MetroMart ADR: do not start yet on this page. The master template remains read-only.

10. Common beginner mistakes

Watch for these traps

- Reading every box instead of defending decisions.
- No rejected alternative.
- Claiming runtime evidence that was only simulated locally.
- Changing the whole architecture when one constraint changes.

11. Stuck / recovery help

Recovery ladder

Use the defense order: goal -> hard requirements -> major flow -> 3 key decisions -> biggest risk -> recovery evidence -> cost/security guardrail -> changed constraint -> revisit decision.

12. Small independent task

MetroMart - close teacher example first

MetroMart: close videos/review/AI solving. CREATE a 5-minute defense. Then change one constraint (for example festival load becomes 8x or inventory freshness becomes 30 seconds) and state exactly which decisions must be revisited.

Save before help

Save `attempt\_01.md` and any copied ADR/diagram before opening Review. A blank or unsaved attempt does not unlock repair.

13. Validate

Validation

Every claim is either requirement-backed, evidence-backed, or clearly labeled assumption. Local checker output is called structural validation only, never real cloud/runtime proof.

14. Teach it back

Explain without notes

Give the 60-second architecture summary to an imaginary interviewer: goal, design, trade-off, failure/recovery, and one revisit trigger.

15. Protected review + changed retry

Failure / repair route

If defense is weak: save attempt -> targeted review -> changed CareLink defense micro-proof -> validation -> reassessment. Do not rewrite the original attempt into a perfect answer.

16. Evidence / mastery + exact next route

Evidence

Save defense notes + diagram + completed ADR + checker output + assumptions/runtime boundary. M17 evidence complete -> G06 Gate A. G06 B/C remain remediation-only.

End-of-module mastery checklist

Competency 1

I can translate requirements into explicit architecture decisions without choosing products first.

Competency 2

I can choose batch/stream/hybrid per flow and explain replay, duplicates, ordering and freshness.

Competency 3

I can defend warehouse/lakehouse/combined boundaries and prevent duplicated truth.

Competency 4

I can compare managed vs build with operating responsibility, TCO and exit risk.

Competency 5

I can find a bottleneck with signals and design a measurable scale validation.

Competency 6

I can name failure domains, blast radius, RTO/RPO and recovery reconciliation.

Competency 7

I can name cost drivers, unit metric, guardrails and SLO trade-offs.

Competency 8

I can design security/governance around classification, identity, least privilege, lineage and audit.

Competency 9

I can use or reject CDC, contracts, Vault, Data Mesh and Kubernetes for requirement-backed reasons.

Competency 10

I can defend the architecture under a changed constraint and distinguish structural/local evidence from live runtime evidence.

Revision / quick reference

Requirements

Goal | flow grain | freshness | correctness | security | RTO/RPO | scale | cost | team/constraints

Decision

Choice | why | rejected alternative | trade-off | evidence | owner | revisit trigger

Streaming

Freshness | event time | ordering | replay | delivery | idempotent sink | backpressure

Storage

Raw | curated | serving | ownership | open format | SQL/Spark | consumers | reconciliation

Scale

Signal -> bottleneck -> action -> load test -> downstream check -> cost check

Reliability

Failure -> blast radius -> detect -> contain -> recover -> replay -> reconcile -> validate

Security

Classify -> identity -> least privilege -> protect -> audit -> lineage -> retain/delete

Defense

Goal -> hard requirements -> 3 decisions -> alternative -> biggest risk -> recovery evidence -> changed constraint

M17 -> G06 exact handoff

Prerequisite evidence

G06 Gate A may begin only after M15 + M16 + M17 learner evidence is complete. For M17, preserve MetroMart attempts, ADR, diagram, validation, review/repair history if used, changed retry if assigned, and FleetStream practical evidence.

Gate state machine

NORMAL ROUTE: G06 Gate A. If Gate A passes K1-K7 with evidence, the G06 family ends. Gate B is remediation-only after a failed/uncertain A plus targeted repair and explicit assignment. Gate C is remediation-only after a second reassessment. Never take B/C as bonus attempts.

What M17 contributes to G06

Primary: K6 architecture trade-offs and K7 integrated defense. Reinforces K1/K2 cloud workload boundaries, K4 security/governance, and K5 reliability/cost. M15 and M16 remain the authoritative technical prerequisites for the cloud/runtime/DataOps competencies.

Runtime truth

M17 static-content seal does not certify live Fabric, GitHub Actions, Terraform, IAM, network, Android/PWA, or learner-PC behavior. Those remain separate runtime/device evidence requirements later in the whole-course release process.